

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3092 Deep Learning, Semestre II 2026



Hoja de Trabajo #2

Roberto Barreda #23354

Guatemala, 5 de octubre de 2026

1. De las RNN a la atención

¿Qué es el "cuello de botella" del seq2seq con RNN y por qué empeora con oraciones largas? ¿Cómo lo resuelve la atención de Bahdanau?

El encoder comprime la entrada en su último estado oculto, un vector de tamaño fijo, y el decoder genera la salida a partir de él. Con oraciones largas ese vector guarda más información en el mismo espacio y lo leído al inicio se pierde. Bahdanau et al. conservan todos los estados del encoder y en cada paso el decoder calcula pesos sobre ellos para formar su propio contexto.

¿Qué diferencia hay entre la atención aditiva de Bahdanau y la de producto punto de Luong? ¿Cuál es más eficiente en hardware moderno?

La aditiva puntúa con una red de una capa, $v^T \tanh(W_1 s + W_2 h)$, y la de Luong con el producto punto $s^T h$ o $s^T W h$. La de producto punto es más eficiente porque todos los puntajes salen de una multiplicación de matrices, que la GPU ejecuta en paralelo.

¿Cómo funciona la analogía de consulta, llave y valor como búsqueda "suave" en un diccionario? ¿De dónde salen Q, K y V?

Un diccionario devuelve el valor de la llave que coincide con la consulta. La atención compara la consulta con todas las llaves, convierte las similitudes en pesos con softmax y devuelve el promedio ponderado de los valores. En self-attention Q, K y V son proyecciones de la misma secuencia. En cross-attention Q viene del decoder y K y V del encoder.

¿Qué limitaciones de las RNN elimina el Transformer al prescindir de la recurrencia?

La RNN calcula el estado t después del $t - 1$ y no se paraleliza en el tiempo. La señal entre tokens separados por n posiciones recorre n pasos y el gradiente se multiplica n veces por la misma matriz, por lo que se desvanece. El Transformer procesa todos los tokens a la vez y conecta cualquier par en un paso.

2. La arquitectura Transformer

Scaled dot-product attention. ¿Por qué se divide entre la raíz de d_k ? ¿Cuál es la varianza del producto punto y qué le pasa a la softmax sin escalar?

Si q y k tienen componentes independientes con media 0 y varianza 1, su producto punto suma d_k productos de varianza 1 y su varianza es d_k . Dividir entre la raíz de d_k la regresa a 1. Sin escalar, la softmax concentra el peso en una llave, su jacobiano tiende a cero y Q y K dejan de aprender, como muestra la Figura 1.

Multi-head attention. ¿Qué se gana con h cabezas de dimensión d_{model}/h ? ¿Cambian los parámetros?

Cada cabeza tiene proyecciones propias y aprende un tipo de relación, como posición, sintaxis o correferencia, en lugar de mezclarlas en una distribución. Los parámetros no cambian porque las proyecciones siguen siendo de $d_{\text{model}} \times d_{\text{model}}$. En el notebook una capa con $d_{\text{model}} = 512$ tiene 1,050,624 parámetros con 1 y con 8 cabezas.

Codificación posicional. ¿Por qué la self-attention no conoce el orden? ¿Cómo se compara la sinusoidal con los embeddings aprendidos y RoPE?

Los pesos dependen del contenido y no de la posición, así que permutar los tokens permuta la salida. La sinusoidal suma senos y cosenos de frecuencias geométricas, $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d})$, sin parámetros. Los embeddings aprendidos de BERT y GPT-2 se limitan a la longitud máxima. RoPE rota q y k según su posición y el producto punto depende de la distancia entre tokens.

Bloque completo. ¿Qué diferencia hay entre Post-LN y Pre-LN?

La residual deja pasar el gradiente, LayerNorm normaliza cada token y la feed-forward transforma cada posición. Post-LN normaliza después de la suma, $\text{LN}(x + F(x))$, y necesita warm-up. Pre-LN normaliza antes, $x + F(\text{LN}(x))$, y entrena redes profundas sin warm-up según Xiong et al. (2020).

Complejidad. ¿Cuál es el costo de la self-attention respecto a n y cómo se reduce?

El tiempo es $O(n^2d)$ y la memoria $O(n^2)$. FlashAttention calcula por bloques en la SRAM con memoria $O(n)$ y resultado exacto, pero el tiempo sigue en $O(n^2)$. Las ventanas de Longformer cuestan $O(nw)$ y pierden relaciones lejanas. La atención lineal cambia la softmax por un producto de funciones de q y k con costo $O(n)$ y aproxima el resultado.

3. Atención en PyTorch y Hugging Face

F.scaled_dot_product_attention

Calcula la atención con el kernel disponible. `attn_mask` es booleana, con `True` en las posiciones permitidas, o flotante y se suma a los puntajes. `dropout_p` aplica dropout a los pesos, `is_causal` aplica la máscara triangular sin construirla y `scale` reemplaza $1 / \sqrt{d_k}$.

nn.MultiheadAttention

Implementa la atención con h cabezas y sus proyecciones. `embed_dim` es d_{model} , `num_heads` debe dividirlo, `batch_first` usa la forma `batch x n x d` y `kdim` y `vdim` aceptan llaves y valores de otra dimensión. En el forward, `key_padding_mask` ignora el relleno, `attn_mask` bloquea pares, `need_weights` devuelve los pesos y `average_attn_weights` los promedia entre cabezas.

nn.TransformerEncoderLayer y generate_square_subsequent_mask

Es un bloque con self-attention, feed-forward, residuales y LayerNorm. `d_model`, `nhead` y `dropout` fijan la dimensión, las cabezas y la tasa de dropout, `dim_feedforward` el tamaño oculto y `norm_first` elige Pre-LN. `generate_square_subsequent_mask(n)` devuelve 0 en la diagonal y debajo y $-\infty$ arriba como máscara causal.

output_attentions en Hugging Face

Con `output_attentions=True` se obtiene una tupla con un tensor de `batch x cabezas x n x n` por capa, 12 de $1 \times 12 \times n \times n$ en BERT y GPT-2 base. Se usa `attn_implementation="eager"` porque SDPA no devuelve los pesos.

4. Ejercicio a mano

Se usan tres tokens con $d_k = 2$. Se calcula QK^T , se divide entre la raíz de 2 = 1.4142, se aplica softmax por fila y se multiplica por V . En la fila 1 los puntajes escalados son 0.7071, 0 y 0.7071. Como $e^{0.7071} = 2.0281$, la suma es 5.0562 y los pesos son 0.4011, 0.1978 y 0.4011. La salida es $0.4011[1, 2] + 0.1978[3, 4] + 0.4011[5, 6] = [3, 4]$. Con máscara causal el token 1 se ve a sí mismo y el token 2 reparte 0.5 entre los dos primeros. El notebook reproduce la Tabla 1 con error menor a $1e-4$.

Tabla 1. Pasos del ejercicio a mano. Cada fila es una consulta con sus puntajes, pesos y salida.

Token	q	k	v	Fila de QK^T	Escalada	Pesos	Salida	Salida causal
1	[1, 0]	[1, 1]	[1, 2]	1, 0, 1	0.7071, 0, 0.7071	0.4011, 0.1978, 0.4011	[3.0000, 4.0000]	[1.0000, 2.0000]
2	[0, 1]	[0, 1]	[3, 4]	1, 1, 0	0.7071, 0.7071, 0	0.4011, 0.4011, 0.1978	[2.5933, 3.5933]	[2.0000, 3.0000]
3	[1, 1]	[1, 0]	[5, 6]	2, 1, 1	1.4142, 0.7071, 0.7071	0.5035, 0.2483, 0.2483	[2.4895, 3.4895]	[2.4895, 3.4895]

5. Verificaciones

Tabla 2. Diferencia máxima entre la implementación propia y PyTorch. Todas están en el orden del redondeo de float32.

Verificación	Error máximo
Propia contra F.scaled_dot_product_attention sin máscara	3.6e-7
Propia contra F.scaled_dot_product_attention con is_causal	2.4e-7
Suma de cada fila de pesos menos 1	1.8e-7
Multi-head propia contra nn.MultiheadAttention, salida y pesos	6.0e-8
Posiciones distintas con generate_square_subsequent_mask	0
Peso sobre la diagonal con máscara causal y en 1,152 matrices de GPT-2	0

6. Escalamiento y costo computacional

Sin escalar, la varianza del producto punto coincide con d_k , la entropía normalizada de la softmax cae de 0.65 a 0.03 y la norma del jacobiano baja de 0.30 a 0.055. Con el escalamiento las tres medidas no dependen de d_k .

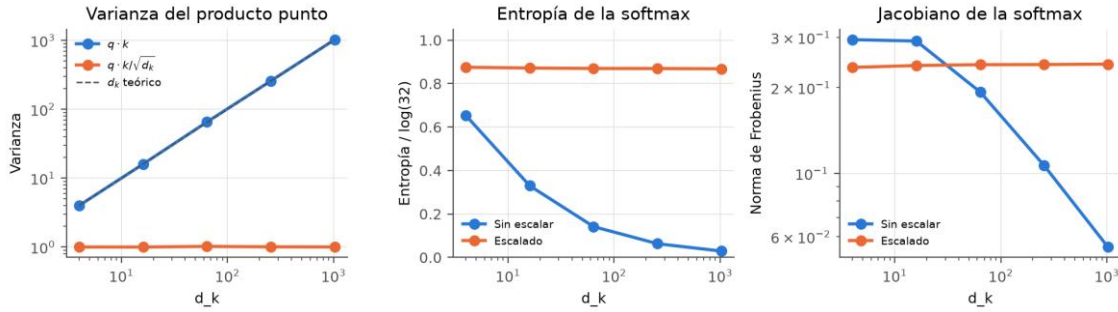


Figura 1. Efecto de dividir entre la raíz de d_k sobre la varianza del producto punto, la entropía de la softmax con 32 llaves y la norma de su jacobiano.

La versión propia guarda la matriz $n \times n$ y su memoria crece con n^2 hasta 8,208 MB en $n = 16,384$. El kernel de memoria eficiente de SDPA procesa por bloques y usa 16 MB. PyTorch para Windows no incluye FlashAttention, por eso se usó ese kernel. El tiempo crece con n^2 en ambos porque las operaciones son las mismas, aunque SDPA tarda 10.8 ms contra 65.7 ms.

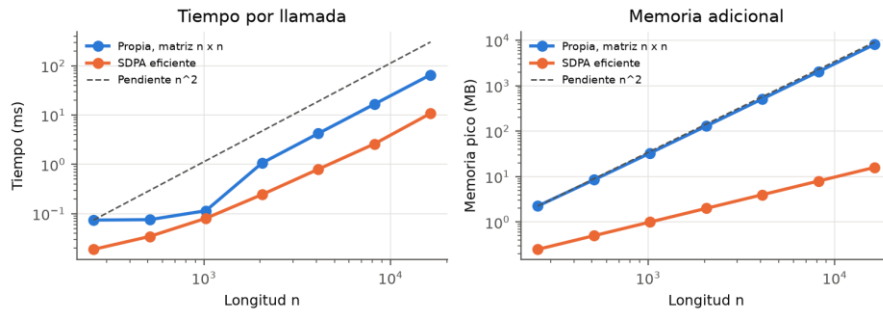


Figura 2. Tiempo y memoria pico de la atención con 8 cabezas, $d = 64$ y float16 en una RTX 4080 Laptop. La línea punteada marca la pendiente n^2 .

7. Mapas de atención en BERT y GPT-2

Para cada cabeza se midió la atención al token anterior, a [CLS] o [SEP], a la puntuación y a otra aparición de la misma palabra, y la Figura 3 muestra la cabeza con el valor mayor en capas distintas. En BERT hay cabezas de token anterior, de [SEP] con 0.87 de la atención, de coma y punto, y una que une cada "cat" y cada "the" con su otra aparición. En GPT-2 la cabeza 11 de la capa 4 pone peso 1.0 en el token anterior y la cabeza 2 de la capa 7 manda la atención al primer token. GPT-2 no tiene cabezas de token siguiente por la máscara causal.

En el par del enunciado, la cabeza 10 de la capa 6 de BERT da 0.28 a "animal" y 0.07 a "street" con tired, y 0.59 a "street" y 0.01 a "animal" con wide. 22 de 144 cabezas cumplen ambos casos. En GPT-2 el peso sobre la diagonal es 0 en todas las matrices. El primer token recibe 0.24 de la atención en la capa 0 y entre 0.69 y 0.83 desde la capa 5, frente a 0.185 de una distribución uniforme. Con otras palabras iniciales el valor queda entre 0.69 y 0.78, así que depende de la posición. Es el attention sink de Xiao et al. (2023), donde el primer token absorbe la atención que sobra porque la softmax obliga a sumar 1. BERT usa [SEP] igual entre las capas 4 y 9.

Las capas profundas atienden de forma más concentrada. En BERT la entropía baja de 2.02 nats en la capa 0 a 1.15 en la capa 7 y sube a 1.39 en la 11. En GPT-2 baja de 1.28 a 0.59 en la capa 9 y es menor que en BERT en todas las capas, en parte por el peso en el primer token. La Figura 4 resume estos tres resultados.

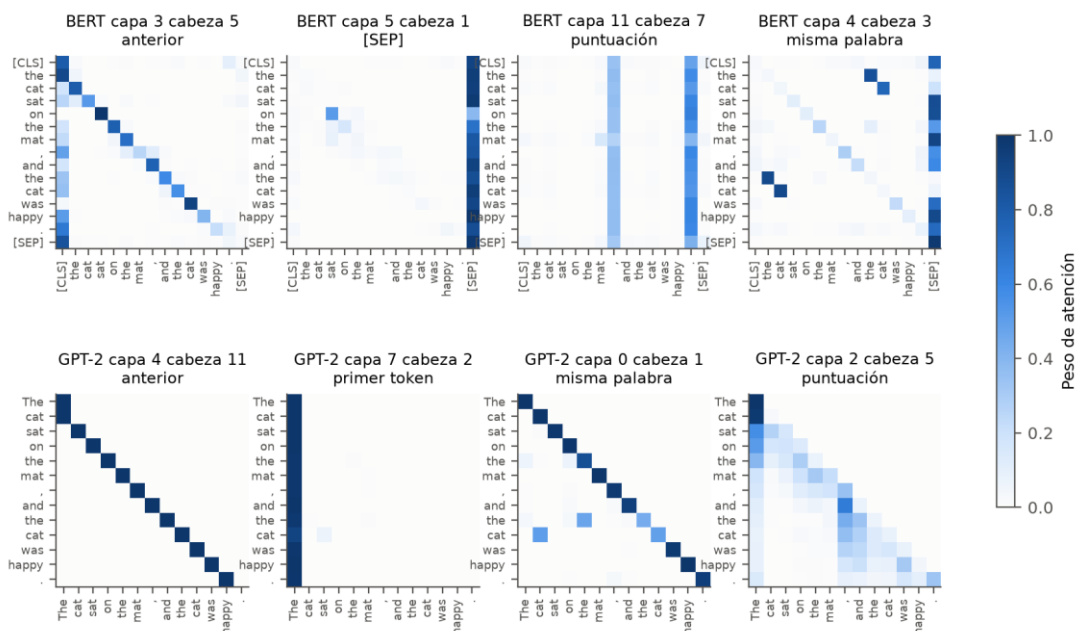


Figura 3. Pesos de cuatro cabezas de BERT, arriba, y de GPT-2, abajo, para la oración The cat sat on the mat, and the cat was happy. Las filas son consultas y las columnas llaves.

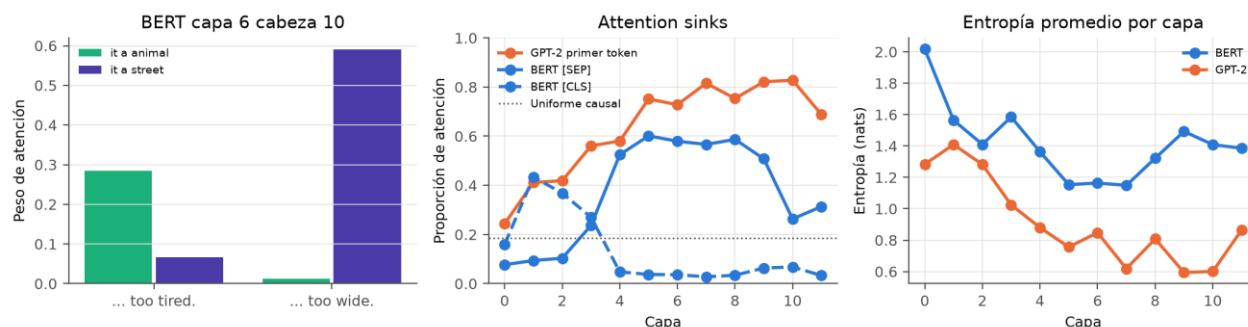


Figura 4. Atención desde "it" en la cabeza de correferencia de BERT, atención al primer token de GPT-2 y a [CLS] y [SEP] de BERT por capa, y entropía promedio por capa en 8 oraciones.

8. Discusión y análisis

¿Por qué la atención permitió entrenar modelos más grandes que las RNN?

La atención procesa todos los tokens con multiplicaciones de matrices, sin dependencia entre pasos, y aprovecha el paralelismo de la GPU. El camino entre dos tokens es de un paso en lugar de n y el gradiente no se desvanece. El costo cuadrático es el precio, pero para 512 a 2048 tokens pesa menos que la secuencialidad y kernels como el de la Figura 2 reducen su memoria.

¿Qué evidencia hay de que las cabezas se especializan? ¿Todas parecen útiles?

La Figura 3 muestra cabezas de token anterior con pesos de 0.61 en BERT y 1.0 en GPT-2, de puntuación, de palabras repetidas y una cabeza de BERT que sigue la correferencia de "it". No todas parecen útiles. En la oración de la Figura 3 el primer token recibe 0.65 de la atención de GPT-2 y [SEP] 0.31 de la de BERT. Clark et al. (2019) ven la atención a [SEP] como una operación nula y Michel et al. (2019) eliminan muchas cabezas sin perder rendimiento.

¿Los pesos de atención explican las decisiones del modelo?

No de forma completa. 22 de 144 cabezas siguen la correferencia y elegir la que confirma la hipótesis es un sesgo. Los pesos dicen de dónde se mezcla la información, no cuánto influye cada token, y las cabezas que envían 0.8 al primer token tienen peso alto sin contenido. Jain y Wallace (2019) muestran que los pesos se correlacionan poco con la importancia por gradientes y que otras distribuciones dan la misma

predicción. Wiegreffe y Pinter (2019) responden que esas distribuciones no surgen al entrenar y que la atención puede ser una explicación plausible, no la única. Los mapas sirven para formar hipótesis que se validan con ablaciones.

¿En qué casos preferirían una RNN o una CNN sobre un Transformer?

Con pocos datos, porque las CNN y las RNN traen sesgos de localidad y orden. En streaming o en dispositivos con poca memoria, donde la RNN procesa cada dato con memoria constante. En audio o series de cientos de miles de pasos, donde las CNN escalan de forma lineal.

¿Qué problema darían documentos de 100,000 tokens y qué alternativas considerarían?

La matriz de puntajes tendría 10^{10} elementos por cabeza y capa, 20 GB en float16, y la versión propia con 8 cabezas necesitaría cerca de 300 GB por capa según la Figura 2. Usaría FlashAttention o el kernel eficiente, que son exactos con memoria lineal, junto con atención por ventanas, RoPE con interpolación de posiciones o fragmentos con recuperación.

9. Conclusiones

1. Dividir entre la raíz de d_k mantiene la varianza de los puntajes en 1 y evita que la softmax se sature.
2. La memoria de la atención estándar crece con n^2 y el kernel eficiente de SDPA la reduce de 8,208 MB a 16 MB con $n = 16,384$.
3. Las cabezas de BERT y GPT-2 se especializan en token anterior, puntuación, palabras repetidas y coreferencia.
4. GPT-2 envía entre 0.69 y 0.83 de la atención al primer token desde la capa 5 sin importar la palabra.
5. Las capas profundas de BERT y GPT-2 atienden de forma más concentrada que las primeras.

10. Referencias

- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR. arXiv:1409.0473.
- Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. arXiv:2004.05150.
- Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). What does BERT look at? An analysis of BERT's attention. BlackboxNLP. arXiv:1906.04341.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. NeurIPS. arXiv:2205.14135.
- Jain, S., & Wallace, B. C. (2019). Attention is not explanation. NAACL. arXiv:1902.10186.
- Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. ICML. arXiv:2006.16236.
- Luong, M. T., Pham, H., & Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. EMNLP. arXiv:1508.04025.
- Michel, P., Levy, O., & Neubig, G. (2019). Are sixteen heads really better than one? NeurIPS. arXiv:1905.10650.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). RoFormer: Enhanced transformer with rotary position embedding. arXiv:2104.09864.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. NeurIPS. arXiv:1706.03762.
- Wiegreffe, S., & Pinter, Y. (2019). Attention is not not explanation. EMNLP. arXiv:1908.04626.
- Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2023). Efficient streaming language models with attention sinks. arXiv:2309.17453.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. (2020). On layer normalization in the transformer architecture. ICML. arXiv:2002.04745.

Repositorio

<https://github.com/bar23354/DL-HDT2>